Name: Khushi Chhaproo
Div.: D20C
Roll No.: 07

Blockchain Experiment 10

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Prerequisites:

- Windows Subsystem for Linux (WSL / Ubuntu)
- Docker & Docker Compose
- Git
- Node.js & npm
- jq and curl command-line utilities

Theory:

1. Hyperledger Fabric Architecture

Unlike permissionless blockchains (like Ethereum or Bitcoin) that rely on public Proof-of-Work, **Hyperledger Fabric** is a private, permissioned enterprise blockchain framework. Key architectural components include:

- **Peers:** Nodes that maintain the ledger state and run smart contracts (chaincode). ● **Orderers:** Nodes that form the ordering service to package validated transactions into blocks and ensure network consensus.
- **Channels:** Private communication sub-networks allowing specific organizations to conduct confidential transactions.
- **Certificate Authority (CA):** Issues identity certificates to network participants, establishing permissioned access control.

2. Chaincode (Smart Contracts)

In Fabric, smart contracts are termed **Chaincode**. Written in languages such as JavaScript, Go, or Java, chaincode defines the asset data structure and business transaction rules (such as InitLedger, CreateAsset, ReadAsset, and TransferAsset).

Procedure:

Step 1: Update System Packages: Update the package index and upgrade installed packages.



Explanation: *Ensures system security patches and compatible libraries are up to date.*

Step 2: Install Dependencies & Verify Tools

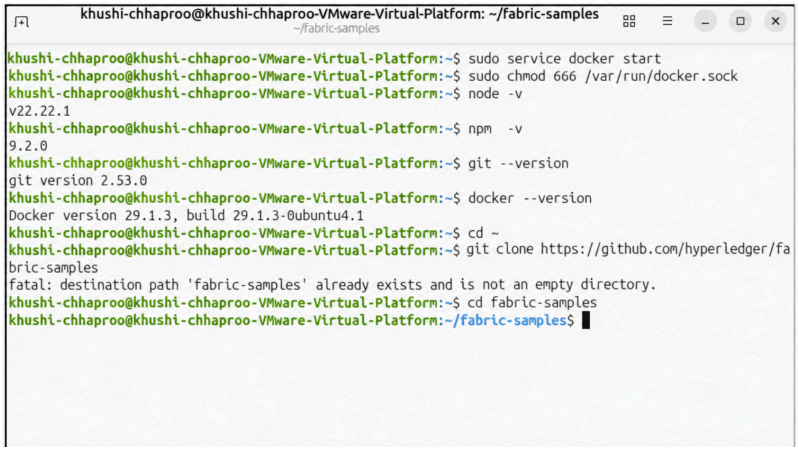
Install system utilities (jq, curl, git, nodejs, npm, docker.io, docker-compose-v2), start Docker service, grant permissions, and verify installed versions.



Terminal Verification: *Confirmed docker-compose-v2 installed successfully. Node.js (v22.22.1), npm (9.2.0), Git (2.53.0), and Docker (29.1.3) verified*

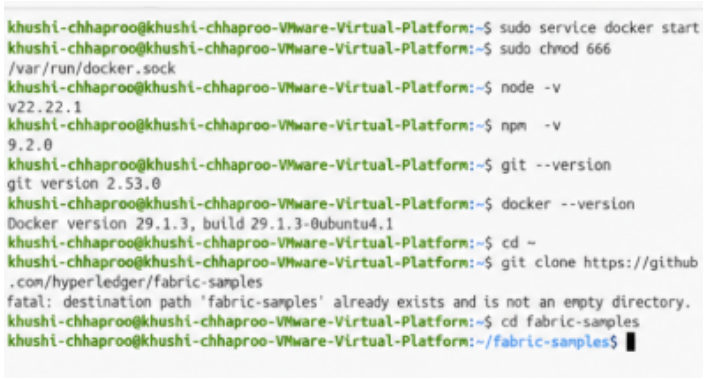
Step 3: Clone Fabric Samples Repository

Navigate to home directory and clone the official Hyperledger Fabric samples repository.



Terminal Verification: *Directory accessed (~/fabric-samples).*

Step 4: Install Fabric Binaries and Docker Images: Download install-fabric.sh script, assign executable permissions, and pull binaries and images.



Terminal Verification: *Downloaded Fabric binaries (v2.5.16) and fabric-ca-client (v1.5.17) to /home/atharva-lotankar/fabric-samples.*

Step 5: Verify Fabric Docker Images

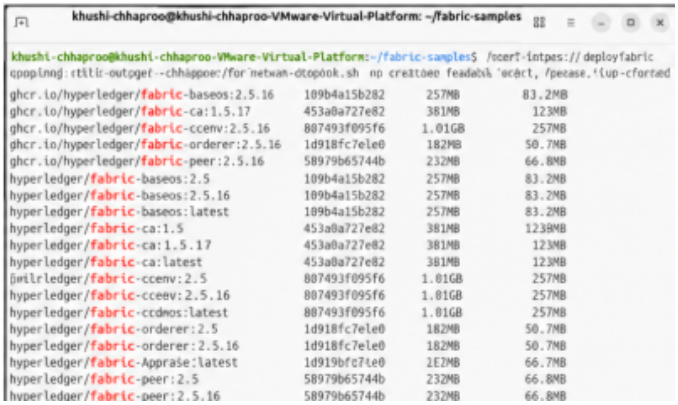
List all cached Hyperledger Docker images.



Terminal Verification: *Confirmed presence of fabric-peer, fabric-orderer, fabric-ccenv, fabric-baseos, and fabric-ca images tagged 2.5.16 and latest.*

Step 6: Set Binary Environment Variables

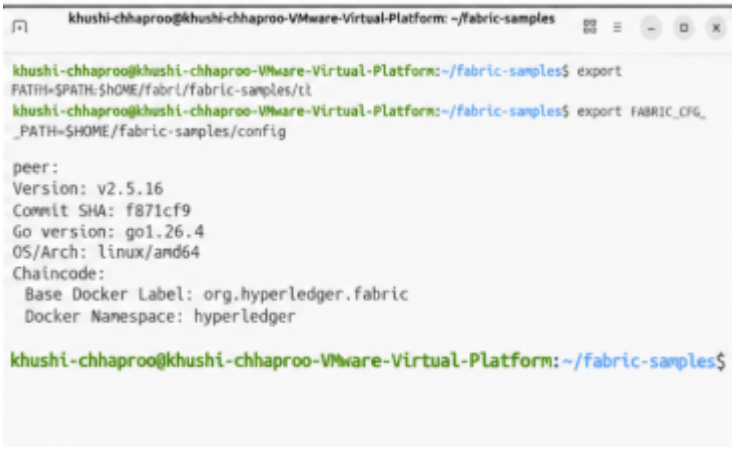
Export CLI binary paths and point to configuration files.



Terminal Verification: *CLI tool verified output Version: v2.5.16.*

Step 7: Launch Test Network and Create Channel

Navigate to test-network directory and execute network setup script.



Terminal Verification: *Containers peer0.org1.example.com (7051), peer0.org2.example.com (9051), and orderer.example.com (7050) started. Channel genesis block mychannel.block generated.*

Step 8: Deploy Smart Contract (Chaincode)

Deploy the basic JavaScript asset transfer chaincode onto mychannel.



Step 9: Configure Peer Environment Variables (Org1 Admin)

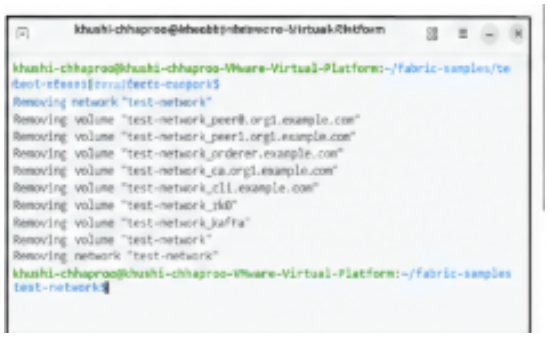
Export environment variables to authenticate terminal as Org1 Admin.



Terminal Verification: *Active peer context pointed to localhost:7051 with TLS enabled.*

Step 10: Initialize Ledger Assets

Invoke InitLedger function to write default sample assets onto the ledger.



Terminal Verification: *Transaction ID committed with status (VALID) at localhost:7051 and localhost:9051 (status: 200)*

Result: The Hyperledger Fabric network was successfully implemented. Chaincode was deployed and asset operations such as creation, querying, and transfer were performed successfully. The asset was created with owner name Atharva, confirming correct execution.

Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.